34.4 TTS（文本转语音）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~98k tokens，输出 ~8k tokens（本节）

文本转语音（Text-to-Speech，TTS）让 Agent 的回复可以变成语音消息发送给用户——这在移动端聊天场景中尤其实用。OpenClaw 的 TTS 模块（`src/tts/tts.ts`，约 1580 行）支持三个 Provider、四种自动模式、Agent 内联指令覆盖，以及超长文本的自动摘要。

34.4.1 TTS 引擎（`src/tts/`）

三 Provider 架构

OpenClaw 支持三个 TTS Provider，按可用性自动降级：

Provider	类型	默认模型/声音	API Key 来源
OpenAI	云端 API	`gpt-4o-mini-tts` / `alloy`	`OPENAI_API_KEY` 或配置
ElevenLabs	云端 API	`eleven_multilingual_v2` / 预设声音	`ELEVENLABS_API_KEY` 或 `XI_API_KEY`
Edge TTS	免费服务	`en-US-MichelleNeural`	无需 Key（默认 Provider）

> **衍生解释 — Edge TTS**
>
> Edge TTS 是微软 Edge 浏览器内置的语音合成服务的非官方 Node.js 封装（`node-edge-tts` 库）。它利用微软认知服务的在线接口，支持数百种语言和声音，无需 API Key，完全免费。由于不需要任何配置，OpenClaw 将其作为默认的 TTS Provider。

Provider 选择的优先级逻辑：

```
// src/tts/tts.ts
export function getTtsProvider(config, prefsPath): TtsProvider {
  // 1. 用户偏好（持久化到 JSON 文件）
  const prefs = readPrefs(prefsPath);
  if (prefs.tts?.provider) return prefs.tts.provider;

  // 2. 配置文件显式指定
  if (config.providerSource === "config") return config.provider;

  // 3. 自动探测（有 Key 的优先）
  if (resolveTtsApiKey(config, "openai")) return "openai";
  if (resolveTtsApiKey(config, "elevenlabs")) return "elevenlabs";
  return "edge";  // 兜底
}
```

四种自动模式

TTS 的触发时机通过 `TtsAutoMode` 控制：

模式	行为
`off`	完全关闭 TTS
`always`	每条回复都生成语音
`inbound`	仅当用户发送了音频/语音消息时生成语音回复
`tagged`	仅当 Agent 输出中包含 `[[tts]]` 指令标签时生成

自动模式的优先级链为：**会话级覆盖 > 用户偏好文件 > 配置文件**。

配置解析

`resolveTtsConfig` 将原始 YAML 配置展开为完整的 `ResolvedTtsConfig` 结构，所有可选字段都填充了默认值：

```
export type ResolvedTtsConfig = {
  auto: TtsAutoMode;              // 自动模式
  mode: TtsMode;                  // "final" 仅最终回复 | "all" 每条都生成
  provider: TtsProvider;
  summaryModel?: string;          // 摘要使用的 LLM 模型
  modelOverrides: ResolvedTtsModelOverrides;  // 指令覆盖策略
  elevenlabs: { ... };            // ElevenLabs 完整配置
  openai: { ... };                // OpenAI 完整配置
  edge: { ... };                  // Edge TTS 完整配置
  maxTextLength: number;          // 硬上限 4096 字符
  timeoutMs: number;              // API 超时 30 秒
};
```

用户偏好持久化

TTS 偏好（开关、Provider、最大长度、是否摘要）通过 JSON 文件持久化，默认路径为 `~/.config/openclaw/settings/tts.json` 。偏好文件的读写使用原子写入（先写临时文件再 `rename` ），避免并发写入导致文件损坏：

```
function atomicWriteFileSync(filePath: string, content: string): void {
  const tmpPath = `${filePath}.tmp.${Date.now()}.${Math.random().toString(36).slice(2)}
  writeFileSync(tmpPath, content);
  renameSync(tmpPath, filePath);  // 原子替换
}
```

34.4.2 三个 Provider 的实现

OpenAI TTS

调用 OpenAI 的 `/v1/audio/speech` 端点：

```
async function openaiTTS(params): Promise<Buffer> {
  const response = await fetch(`${getOpenAITtsBaseUrl()}/audio/speech`, {
    method: "POST",
    headers: { Authorization: `Bearer ${apiKey}`, "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "gpt-4o-mini-tts",    // 或 tts-1, tts-1-hd
      input: text,
      voice: "alloy",              // 9 种声音可选
      response_format: "opus",     // Telegram 用 opus，其他用 mp3
    }),
    signal: controller.signal,     // AbortController 超时控制
  });
  return Buffer.from(await response.arrayBuffer());
}
```

支持通过 `OPENAI_TTS_BASE URL` 环境变量指向自定义 TTS 端点（如本地部署的 Kokoro、LocalAI），此时模型和声音的白名单校验自动放宽。

可用声音：`alloy`、`ash`、`coral`、`echo`、`fable`、`onyx`、`nova`、`sage`、`shimmer`（9 种）。

ElevenLabs TTS

ElevenLabs 提供高质量的人声克隆和多语言合成，API 参数最为丰富：

```
async function elevenLabsTTS(params): Promise<Buffer> {
  const url = `${baseUrl}/v1/text-to-speech/${voiceId}?output_format=${outputFormat}`;
  const response = await fetch(url, {
    method: "POST",
    headers: { "xi-api-key": apiKey, "Content-Type": "application/json" },
    body: JSON.stringify({
      text,
      model_id: "eleven_multilingual_v2",
      seed: seed,                           // 可选：固定随机种子
      apply_text_normalization: "auto",     // 文本归一化模式
      language_code: "en",                  // ISO 639-1 语言代码
      voice_settings: {
        stability: 0.5,           // 0-1，越高越稳定
        similarity_boost: 0.75,   // 0-1，声音相似度
        style: 0.0,               // 0-1，表现力
        use_speaker_boost: true,  // 增强说话者特征
        speed: 1.0,               // 0.5-2.0 语速
      },
    }),
  });
  return Buffer.from(await response.arrayBuffer());
}
```

Edge TTS

Edge TTS 是最简单的 Provider，通过 `node-edge-tts` 库直接生成音频文件：

```
async function edgeTTS(params): Promise<void> {
  const tts = new EdgeTTS({
    voice: "en-US-MichelleNeural",
    lang: "en-US",
    outputFormat: "audio-24khz-48kbitrate-mono-mp3",
    rate: "+0%",       // 语速调整
    pitch: "+0Hz",     // 音高调整
    volume: "+0%",     // 音量调整
    proxy: undefined,  // 可选 HTTP 代理
  });
  await tts.ttsPromise(text, outputPath);
}
```

Edge TTS 支持通过输出格式自动推断文件扩展名——如果格式字符串包含 `ogg` 、 `opus` 、 `webm` 、 `wav` 等关键词，会生成对应扩展名的文件。当自定义格式失败时，自动回退到默认的 MP3 格式。

通道适配

不同聊天平台对音频格式有不同偏好。TTS 输出格式根据目标通道动态调整：

通道	OpenAI 格式	ElevenLabs 格式	扩展名	语音兼容
Telegram	`opus`	`opus_48000_64`	`.opus`	✓（语音气泡）
其他	`mp3`	`mp3_44100_128`	`.mp3`	✗（文件附件）
电话（VoIP）	`pcm` (24kHz)	`pcm_22050` (22kHz)	-	-

Telegram 渠道使用 Opus 编码，因为 Telegram 的语音消息气泡只支持 OGG Opus 格式。

34.4.3 Agent 内联指令

Agent 可以在输出文本中嵌入 `[[tts:...]]` 指令来控制语音合成的各项参数：

指令语法

参数指令： `[[tts:key=value key2=value2]]`

```
[[tts:provider=elevenlabs voice_id=pMsXgVXv3BLzUgSXRplE speed=1.2]]
这段话将使用 ElevenLabs 的指定声音和 1.2 倍速朗读。
```

文本块指令： `[[tts:text]]...[[/tts:text]]`

```
[[tts:text]]
这是要朗读的内容，与显示给用户的文本不同。
[[/tts:text]]
这段文字只会显示给用户，不会被朗读。
```

可覆盖参数

指令键	作用	约束
`provider`	切换 Provider	openai / elevenlabs / edge
`voice` / `openai_voice`	OpenAI 声音	9 种预设声音之一
`voiceId` / `voice_id`	ElevenLabs 声音 ID	10-40 位字母数字
`model` / `modelId`	模型 ID	根据 Provider 自动路由
`stability`	稳定性	0-1
`similarity`	相似度增强	0-1
`style`	风格表现力	0-1
`speed`	语速	0.5-2.0
`seed`	随机种子	0-4294967295
`language`	语言代码	ISO 639-1 两字母
`normalize`	文本归一化	auto / on / off

覆盖策略

管理员可以通过 `modelOverrides` 配置精细控制 Agent 被允许覆盖哪些参数：

```yaml
messages:
  tts:
    modelOverrides:
      enabled: true          # 总开关
      allowText: true        # 允许 [[tts:text]]
      allowProvider: true    # 允许切换 Provider
      allowVoice: true       # 允许切换声音
      allowModelId: false    # 禁止切换模型
      allowVoiceSettings: true
      allowSeed: false
```

34.4.4 超长文本摘要

当 Agent 的回复超过最大长度限制（默认 1500 字符）时，TTS 模块有两种处理策略：

1. 摘要模式（默认开启）：调用 LLM（默认使用 Agent 的主模型）生成一段不超过 `maxLength` 字符的摘要，再将摘要转为语音：

```
async function summarizeText(params): Promise<SummarizeResult> {
  const { ref } = resolveSummaryModelRef(cfg, config);
  const model = resolveModel(ref.provider, ref.model, undefined, cfg);

  const res = await completeSimple(model, {
    messages: [{
      role: "user",
      content: `Summarize the text to approximately ${targetLength} characters...`
    }]
  }, { apiKey, maxTokens: Math.ceil(targetLength / 2), temperature: 0.3 });

  return { summary: extractText(res), latencyMs: Date.now() - start };
}
```

2. 截断模式（摘要关闭时）：简单地截断文本到 `maxLength - 3` 字符并追加 `...` 。

完整的 TTS 应用流程

`maybeApplyTtsToPayload` 是 TTS 与消息系统的集成入口，流程如下：

```
Agent 回复 payload
  ↓ 检查自动模式（off/always/inbound/tagged）
  ↓ 解析 [[tts:...]] 指令 → 提取覆盖参数 + 清理文本
  ↓ 跳过条件检查（已有媒体附件？文本太短？包含 MEDIA:？）
  ↓ 超长文本处理（摘要 or 截断）
  ↓ textToSpeech() → 尝试 Provider 链（primary → fallback1 → fallback2）
  ↓ 保存音频到临时文件（5 分钟后自动清理）
  ↓ 返回更新后的 payload（mediaUrl = 音频路径，audioAsVoice = true/false）
```

Provider 降级链

`textToSpeech` 按优先级链尝试所有 Provider。当前 Provider 失败（API Key 缺失、超时、错误）时，自动尝试下一个：

```
const providers = resolveTtsProviderOrder(provider);
// 例如 primary=openai → ["openai", "elevenlabs", "edge"]

for (const provider of providers) {
  try {
    // 尝试当前 Provider
    const audioBuffer = await callProvider(provider, text, config);
    writeFileSync(audioPath, audioBuffer);
    return { success: true, audioPath, provider };
  } catch (err) {
    lastError = `${provider}: ${err.message}`;
    // 继续尝试下一个
  }
}
return { success: false, error: lastError };
```

临时文件管理

TTS 生成的音频文件保存在系统临时目录（ `/tmp/tts-xxx/` ），通过 `scheduleCleanup` 注册一个 5 分钟后的定时器自动删除：

```
function scheduleCleanup(tempDir: string, delayMs = 5 * 60 * 1000): void {
  const timer = setTimeout(() => {
    rmSync(tempDir, { recursive: true, force: true });
  }, delayMs);
  timer.unref();  // 不阻止进程退出
}
```

系统提示词注入

当 TTS 启用时， `buildTtsSystemPromptHint` 会向 Agent 的系统提示词中注入 TTS 相关提示：

```
Voice (TTS) is enabled.
Keep spoken text ≤1500 chars to avoid auto-summary (summary on).
Use [[tts:...]] and optional [[tts:text]]...[[/tts:text]] to control voice/expressivene
```

这让 Agent 知道自己正在语音模式下工作，从而生成更适合朗读的文本。

本节小结

1. **三 Provider 架构**：OpenAI（高质量）、ElevenLabs（声音克隆）、Edge TTS（免费兜底），按优先级链自动降级。
2. **四种自动模式**：off/always/inbound/tagged，通过会话 > 偏好文件 > 配置三层优先级控制。
3. **通道自适应**：Telegram 使用 Opus 格式（语音气泡），其他平台使用 MP3（文件附件），电话使用 PCM 原始音频。
4. **Agent 内联指令**：通过 `[[tts:...]]` 标签控制 Provider、声音、语速等参数，由管理员配置的覆盖策略决定哪些参数可被覆盖。
5. **超长文本摘要**：超过限制的文本通过 LLM 摘要或截断处理后再转语音。
6. **临时文件管理**：音频文件保存到 `/tmp/` ，5 分钟后自动清理，定时器设置 `unref()` 不阻止进程退出。